

11. ПОДГОТОВКА К ТЕСТИРОВАНИЮ ПРОДУКТА

11.1. Тестирование программного продукта

Все более распространенным становится заблуждение, согласно которому тестировщики занимаются тем, что просто нажимают на кнопки и вводят рандомную информацию в разные поля программы. На самом деле это не так, если бы тестировщики хаотично нажимали на кнопки и вводили случайные данные, то результаты тестирования никакой ценности для разработчика не принесли бы. Результаты представляли бы собой неструктурированную информацию, из которой невозможно получить представление о том, насколько качественным получился продукт и насколько удобен он для пользователей. У тестировщиков всегда есть стратегия работы, план, который позволяет получить объективное описание актуального состояния продукта.

Также считается, что тестировщики ответственны за качество ПО. На самом деле, ответственность за качество разработки продукта несет вся команда. Тестировщики же помогают улучшать качество разработки, а также выявляют проблемы на ранних стадиях.

Тестирование программного обеспечения является неотъемлемой частью создания программного продукта. От того, насколько досконально проведены тесты, зависит то, как скоро проект будет сдан окончательно, и будет ли необходимость впоследствии устранять ошибки. Тестирование программного продукта на разных стадиях создания — залог качественного выполнения заказа.

11.2. Тестовый случай или тестовый сценарий (test case)

Тестовый случай или тестовый сценарий (test case) — артефакт, описывающий совокупность этапов, конкретных условий и параметров, необходимых для проверки реализации тестируемой функции или её части.

У стандартного тестового случая есть 5 частей, то есть 5 атрибутов (рис. 11.1):

1. **Порядковый номер тестового сценария**
2. **Название тестового сценария.** Из него должно быть понятно, в чем суть тест кейса.
3. **Предусловия тестового сценария.** Это условия, которые необходимы для проведения тест кейса. Они должны быть выполнены еще до запуска тест кейса. Допустим: компания сдает самокаты в поминутную аренду. Нужно провести тест кейс функции, которая уведомляет пользователя о том, что заряд аккумулятора самоката. Предусловием тест кейса будет то, что самокат должен находиться в состоянии аренды.

4. **Порядок действий** в тест кейсе и описания действий в тестовом сценарии.

5. **Ожидаемый результат тестового сценария**

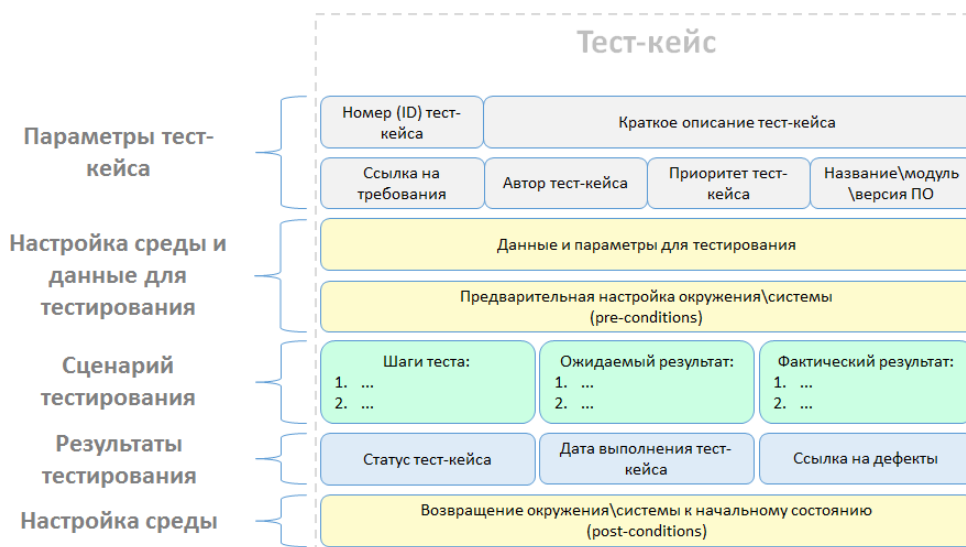


Рисунок 11.1. Атрибуты тестового сценария

Пример заполненного тестового сценария представлен в табл. 11.1.

Таблица 11.1. Пример заполненного тестового сценария

Название:	Кейс – прохождение тренажера	
Функция:	Прохождение теста, основанного на разработанной методологии “Компания”	
Действие	Ожидаемый результат	Результат теста: · пройден · провален · заблокирован
Предусловие:		
Авторизация в ЛК	ЛК – открыт и функционирует согласно с ТЗ	пройден
Шаги теста:		
Запустить тренажер	Выводится окно приветствия	пройден
Нажать на “Начать”	Выводится окно “Приветствие”	пройден
Нажать кнопку “Начать”	Выводится тестовое задание	пройден
Выбрать ответ	Ответ подсвечивается, а кнопка “Далее” становится активной	пройден
Нажать “Далее”	Выводится следующий вопрос с оформлением	пройден
Завершить тест	Сообщается о прохождении и высылаются результаты на почту	провален
Нажать на кнопку “Вернуться в ЛК”	Переход на страницу с тренажерами	пройден

Тестовые сценарии разделяются по ожидаемому результату на **позитивные** и **негативные**:

- **позитивный тест кейс** использует только корректные данные и проверяет, что приложение правильно выполнило вызываемую функцию;
- **негативный тест кейс** оперирует как корректными, так и некорректными данными (минимум 1 некорректный параметр) и ставит целью проверку исключительных ситуаций (срабатывание валидаторов), а также проверяет, что вызываемая приложением функция не выполняется при срабатывании валидатора.

11.3. Этапы тестирования

В работе важно соблюдать все этапы тестирования, чтобы выпустить качественный и полностью функционирующий продукт. Этапы тестирования программного обеспечения включают:

1. **Проектирование тестирования:** Определение цели и объема тестирования, разработка тестового плана, определение ресурсов, формирование расписания и списка требований для тестирования.
2. **Анализ требований и создание тестовых случаев:** Анализ требований к ПО и разработка тестовых случаев. Тестовые случаи описывают шаги, выполняющиеся для проверки функциональных и нефункциональных требований.
3. **Подготовка тестового окружения:** Создание необходимых условий для проведения тестирования, включая установку ПО, настройку тестовых данных, тестовых средств и инструментов. (Виды тестовых сред: среда разработчика, среда тестирования, интеграционная среда, предпрод и продакшн среда)
4. **Выполнение тестов:** Проведение тестирования согласно разработанным тестовым случаям. Формирование чек-листа на основе выполнения тестирования.
5. **Регистрация и отслеживание поломок:** При выявлении ошибок происходит формирование дефектного отчета. В отчете указывается описание проблемы, шаги для воспроизведения, приоритет и серьезность. Дефекты отслеживаются и передаются разработчикам для исправления.
6. **Анализ результатов тестирования:** Оценка выполнения требований как функциональных, так и нефункциональных, обнаружения дефектов, покрытия тестами и эффективность тестирования.
7. **Завершение и отчетность.** Формирование отчета о выполненном тестировании, включающим информацию о проведенных тестах, обнаруженных

дефектах, выполнении требований и других важных аспектах и передача отчета заинтересованным сторонам, например руководителю проекта или заказчику.

11.4. Типы тестирования

Существует большое количество различных типов тестирования, которые представлены на рисунке 11.2.

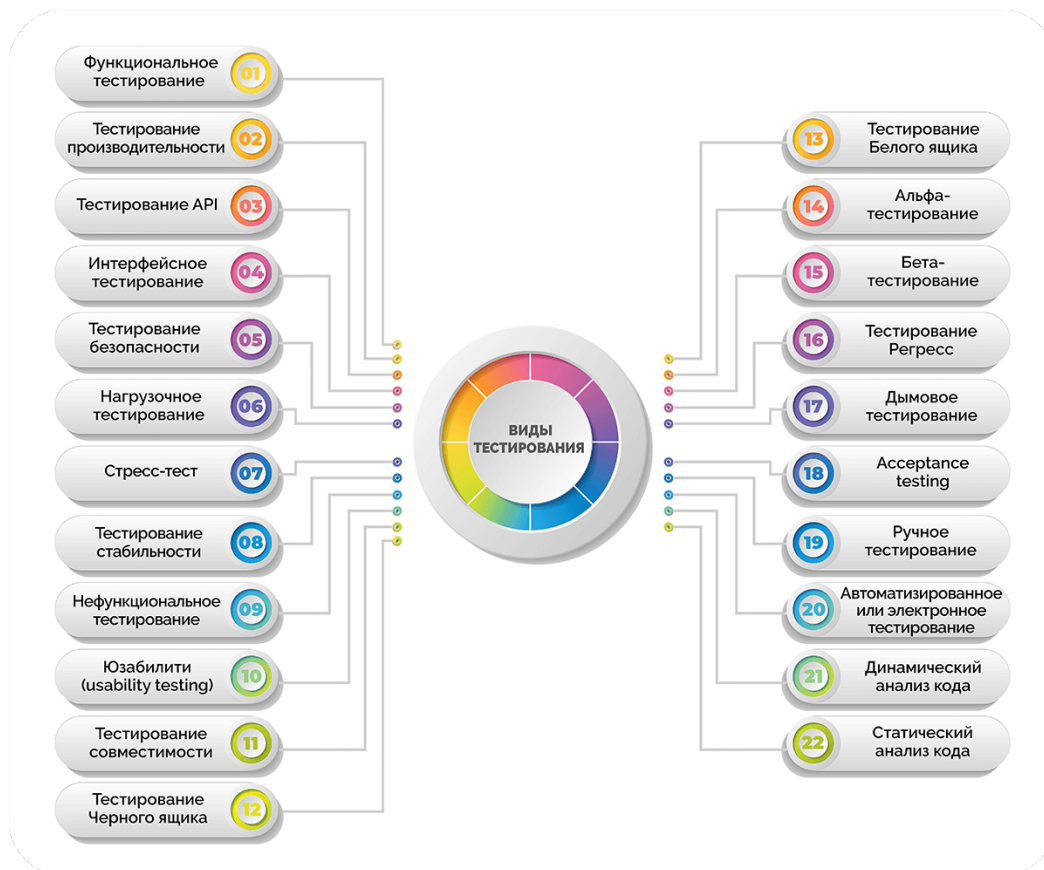


Рисунок 11.2. Виды тестирования

Для тестирования важно различать уровни тестирования, то есть различные ступени или подходы к тестированию ПО, которые обычно выполняются последовательно. Основные уровни тестирования:

1. Модульное тестирование: Проверяются отдельные модули или компонентное наполнение ПО на корректность их функционирования. Тестовые случаи проводятся над отдельными функциями, процедурами или классами. (*Примечание:* в данном случае тестовый случай состоит из одного шага, то есть самой функции (модуля), который подвергается проверки).

2. Интеграционное тестирование: Проверяется взаимодействие и взаимосвязь между различными модулями или компонентами ПО.

3. Системное тестирование: Проводится на интегрированной системе в целом. Проходит проверка функциональности, производительности и поведения системы в соответствии с требованиями и ожиданиями.

4. **Приемочное тестирование:** Проверка соответствия ПО конечным требованиям заказчика или пользователя. Для подтверждения готовности продукта к использованию от заказчика.

5. **Регрессионное тестирование:** Тестирование после внесения изменений в готовое ПО для обнаружения новых дефектов или нежелательных побочных эффектов, возникших в результате внесенных изменений.

6. **Альфа- и бета-тестирование:** Тестирование перед релизом продукта и выполняются в контролируемой среде разработчика (альфа-тестирование) и в реальной среде пользователя (бета-тестирование).

11.5. Check-list (контрольный список задач)

Check-list (контрольный список задач) – список тестовых проверок, которые необходимо выполнить в тестируемой системе. Простыми словами — это перечисление тест-кейсов без деталей (без последовательных шагов и их результатов), главная цель которого запомнить, что мы проверяем и не дать уйти с намеченного пути. Также чек-лист может выступать в роли отчета по проведенному тестированию.

Основные характеристики чек-листа:

1. **Структурированность:** Чек-лист должен состоят из упорядоченных тестовых случаев (как для функциональных, так и для нефункциональных требований), которые необходимо выполнить для полной проверки программного продукта.

2. **Полнота и покрытие:** Чек-лист включает в себя все необходимые аспекты, функции или случаи использования, которые требуется протестировать, чтобы обеспечить высокое качество продукта.

3. **Конкретность:** Каждый элемент в чек-листе должен быть конкретным и понятным, чтобы тестировщик мог точно выполнить задачу или проверить критерий.

4. **Отслеживаемость:** Чек-лист позволяет отслеживать выполнение задач и контролировать прогресс тестирования.

5. **Адаптируемость:** Чек-лист может быть адаптирован под конкретные потребности проекта, включая новые функции, изменения в требованиях или особенности окружения.

Обычно чек-лист включает в себя следующие столбцы:

1. **№:** Этот столбец предназначен для нумерации конкретных тестовых сценариев. Нумерация помогает легко идентифицировать каждый тест и отслеживать его выполнение.

2. **Наименование тестового случая:** В этом столбце указывается краткое и информативное наименование каждого тестового случая. Наименование должно четко отражать то, что будет тестироваться или проверяться в рамках данного теста.

3. **Действия/Входные данные:** В этом столбце указываются действия, которые необходимо выполнить, либо входные данные, которые необходимо использовать для проведения теста. Это может включать в себя ввод определенных данных, выполнение конкретных действий в системе, навигацию по интерфейсу и т.д.

4. **Ожидаемый результат:** Здесь указывается ожидаемый исход тестового сценария или шага. Это то, что должно произойти, если функциональность работает корректно.

5. **Фактический результат:** В этом столбце регистрируется реальный результат выполнения теста. Тестировщик должен записать фактический результат и сравнить его с ожидаемым результатом.

6. **Примечание:** Этот столбец предоставляет возможность добавить дополнительную информацию или комментарии по результатам тестирования. Здесь можно указать, например, какие-то особенности или проблемы, которые были замечены в процессе выполнения теста, требования к конфигурации окружения, использованные тестовые данные и так далее.

Примеры того, что можно добавить в примечание:

- Описание окружения, на котором проводилось тестирование (например, версии операционных систем, браузеров, разрешение экрана и т.д.).
- Ссылки на баг-репорты в системе управления ошибками.
- Список использованных тестовых данных и тестовых случаев.
- Краткие инструкции по воспроизведению найденных проблем.
- Список выполненных действий перед тестированием (например, очистка кэша, обновление страницы и т.д.).

Пример чек-листа представлен в табл. 11.2.

Таблица 11.2. Пример чек-листа тестирования

№	Наименование тестового случая	Действия или Входные данные	Ожидаемый результат	Фактический результат	Примечание
1	Вход в систему (позитивный)	Логин: user@example.com, Пароль: 123456	Успешный вход в систему	Успешный вход в систему	Применение инструментов автоматизированного тестирования веб-интерфейса, например Selenium. Создание и запуск модульных тестов, написанных с использованием подходящего фреймворка.

2	Вход в систему (негативный)	Логин: user@example.com, Пароль: неправильный	Вывод сообщения «Ошибка аутентификации»	Выведено сообщение «Ошибка аутентификации»	Применение инструментов автоматизированного тестирования веб-интерфейса, например Selenium.
3	Оформление заказа (позитивный)	Выбранный товар, Адрес: ул. Пушкина, 10, Способ оплаты: Кредитная карта	Заказ успешно оформлен	Заказ успешно оформлен	Создание и запуск модульных тестов, проверяющих корректность адреса доставки, наличие товара в корзине, есть способ оплаты, корректность суммы заказа. Выполнение ручного тестирования для веб-интерфейса.
4	Оформление заказа (негативный)	Пустая корзина	Ошибка: корзина пуста. Выведено сообщение/изменение интерфейса для пользователя.	Ошибка: корзина пуста. Выведено сообщение/изменение интерфейса для пользователя.	Выполнение автоматизированного тестирования веб-интерфейса системы, например Selenium IDE.
5	Фильтрация товаров (позитивный)	Категория: Электроника, Цена: от \$100 до \$500	Товары отфильтрованы в соответствии с выбранными критериями	Товары отфильтрованы в соответствии с выбранными критериями	Проведение авторизованного/ручного тестирования, проверяющего корректность фильтрации данных относительно запроса, например модульное тестирование, интеграционное и т.д.
6	Время доступа к системе	Вход в систему разных устройств и мест	Доступ к системе в течение X секунд	Результаты измерений времени доступа	Применение следующих инструментов, таких как Apache JMeter, LoadRunner, Gatling, Locust и др.
7	Отказоустойчивость	Нагрузочное тестирование, имитация сбоев	Система выдерживает нагрузку и сбои без потери функциональности	Результаты нагрузочного тестирования	Проведение нагрузочного тестирования и имитации сбоев при использовании специальных инструментов (Apache JMeter, LoadRunner, Gatling, Locust и др.)

8	Безопасность	Тестирование на проникновение, проверка стандартов безопасности	Система защищена от угроз безопасности соответствует стандартам	Результаты тестирования на проникновение и проверки безопасности	Применение автоматизированных инструментов для обнаружения уязвимостей (OWASP ZAP или Burp Suite). Проведение ручного тестирования специалистом по информационной безопасности.
9	Масштабируемость	Добавление большого количества товаров/пользователей	Система масштабируется без потери производительности	Результаты тестирования масштабируемости	Результаты тестирования масштабируемости должны включать данные о производительности системы при различных объемах данных или нагрузке, а также оценку ее способности масштабироваться без потери производительности. Применение инструментов Apache JMeter или Locust.
10	Совместимость	Проверка работоспособности на различных платформах и устройствах	Система корректно работает на различных платформах и устройствах	Результаты тестирования совместимости	Применение инструментов таких как BrowserStack, CrossBrowserTesting или Appium. Проведение ручного тестирования в виде запуска системы в различных браузерах, ОС.

Примечание: в чек-листе необходимо добавить тестовые случаи для всех функциональных и нефункциональных требований, сформированных в первом блоке заданий.

11.6. Дополнение матрицы требований

После того, как будет проведено тестирование необходимо занести итоговые результаты в матрицу требований (табл. 11.3). В матрице требований добавляется столбец “Результат тестирования”.

Если в ходе разработки изменились функциональные или нефункциональные требования, то необходимо внести изменения в матрицу требований.

Таблица 11.3. Дополненная матрица требований

№	Требование	Суть	Критерий проверки	Компоненты архитектуры	Результат тестирования
1	Мобильное приложение клиента				
1.1	Регистрация пользователя	Приложение должно иметь функцию регистрации нового пользователя	Регистрация нового пользователя	Ввод данных реализован на устройстве клиента, обработка происходит на сервере, хранение в единой базе данных	Пользователь зарегистрирован успешно (Требование выполнено)
...	...	...	...	...	...

Примечание: для внесения большой матрицы требований (со всеми дополнениями) в отчет, лучше будет перевернуть страницу в альбомный формат.

11.7. Задание

1. Создать не менее 10 тестовых случаев и оформить согласно таблице 11.1.
2. Описать выбранные инструменты для проведения тестирования.
3. Провести тестирование всех функциональных и нефункциональных требований, сформированных в первом блоке.
4. Результаты проведенного тестирования оформить в виде чек-листа согласно таблице 11.2.
5. Дополнить матрицу требований столбцом “Результат тестирования” и заполнить данными.

11.8. Список литературы о проведении тестирования

1. <https://habr.com/ru/companies/itsumma/articles/682022/>
2. <https://www.intervolga.ru/blog/support/test-surka-testiruem-s-selenium-ide/>
3. <https://habr.com/ru/articles/456090/>
4. <https://habr.com/ru/companies/eurostudio/articles/109010/>
5. <https://habr.com/ru/companies/otus/articles/790780/>
6. <https://hackr.io/blog/top-10-open-source-security-testing-tools-for-web-applications>
7. <https://habr.com/ru/companies/simbirsoft/articles/566862/>